The `fetchCityWeatherData` function takes a `SearchResultItemType` object as an argument, constructs a URL with the latitude and longitude of the item, and sends a `fetch` request to the OpenWeatherMap API. Upon receiving the response, it converts the response into JSON, creates a new `CityWeather` object with the JSON data, and returns it.

Now, `fetchCityWeather` can be updated as follows:

```
const useFetchCityWeather = () => {
  //...
  const fetchCityWeather = (item: SearchResultItemType) => {
    return fetchCityWeatherData(item).then((cityWeather) => {
      setCities([cityWeather, ...cities]);
    });
  };
  //...
}
```

The `useFetchCityWeather` Hook now contains a `fetchCityWeather` function that calls `fetchCityWeatherData` with a given `SearchResultItemType` item. When the promise resolves, it receives a `CityWeather` object and then updates the state cities by adding the new `CityWeather` object at the beginning of the existing cities array.

Next, in the `App` component, we can use `useEffect` to hydrate the `localStorage` data and send requests for the actual weather data:

```
useEffect(() => {
  const hydrate = async () => {
    const items = JSON.parse(localStorage.getItem("favoriteItems") ||
     "[]");

    const promises = items.map((item: any) => {
      const searchResultItem = new SearchResultItemType(item);
      return fetchCityWeatherData(searchResultItem);
    });

    const cities = await Promise.all(promises);
    setCities(cities);
  };

  hydrate();
}, []);
```

In this code snippet, a `useEffect` Hook triggers a function named `hydrate` when the component mounts thanks to the empty dependency array, `[]`.

Inside `hydrate`, firstly, it retrieves a stringified array from `localStorage` under the `favoriteItems` key, parsing it back to a JavaScript array, or defaulting to an empty array if the key doesn't exist. Then, it maps through this items array, creating a new instance of `SearchResultItemType` for each item, which it passes to the `fetchCityWeatherData` function. This function returns a promise, which is collected into an array of promises.

Using `Promise.all`, it waits for all these promises to resolve before updating the state with `setCities`, populating it with the fetched city weather data. Finally, `hydrate` is called within the `useEffect` to execute this logic upon component mounting.

Finally, to save an item in `localStorage` when a user clicks an item, we need a bit more code:

```
const onItemClick = (item: SearchResultItemType) => {
  setTimeout(() => {
    const items = JSON.parse(localStorage.getItem("favoriteItems") ||
"[]");

    const newItem = {
      name: item.city,
      lon: item.longitude,
      lat: item.latitude,
    };

    localStorage.setItem(
      "favoriteItems",
      JSON.stringify([newItem, ...items], null, 2)
    );
  }, 0);

  return fetchCityWeather(item);
};
```

Here, the `onItemClick` function accepts an argument item of the `SearchResultItemType` type. Inside the function, `setTimeout` is utilized with a delay of `0` milliseconds, essentially deferring the execution of its contents until after the current call stack has cleared – so, the UI will not be blocked.

Within this deferred block, it retrieves a stringified array from `localStorage` under the `favoriteItems` key, parsing it back to a JavaScript array, or defaulting to an empty array if the key doesn't exist. Then, it creates a new object, `newItem`, from the item argument, extracting and renaming some properties.

Following this, it updates the `favoriteItems` key in `localStorage` with a stringified array containing `newItem` at the beginning, followed by the previously stored items.

Outside of `setTimeout`, it calls `fetchCityWeather` with the item argument, fetching weather data for the clicked city, and returns the result of this call from `onItemClick`.

Now, when we inspect `localStorage` in our browser, we will see that the objects are listed in JSON format and that the data will persist until users explicitly clean it:

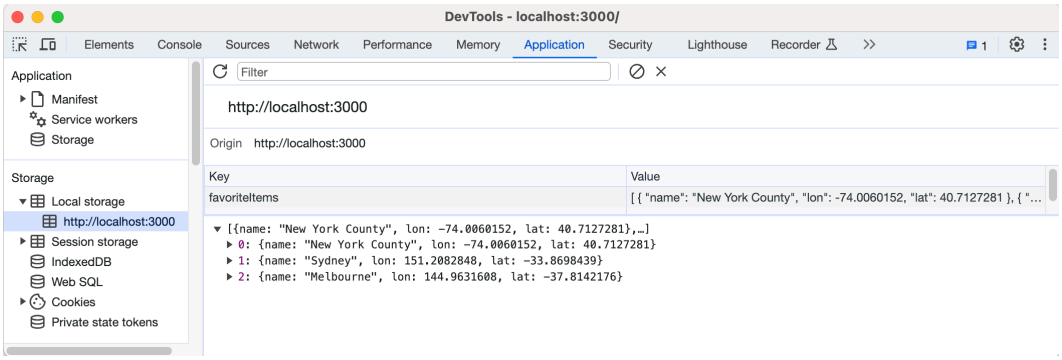


Figure 12.8: Using data from local storage

Excellent job! Everything is now functioning well, and the code is in a robust shape that's easy to build upon. Moreover, the project structure is intuitive, facilitating easy navigation and file location whenever we need to implement changes.

This chapter was quite extensive and was packed with insightful information, serving as a great recapitulation of the knowledge you've acquired thus far. While it would be engaging to continue adding more features together, I believe now is an opportune time for you to dive in and apply the concepts and techniques you've gleaned from this book. I'll entrust the enhancement task to you, confident that you'll make commendable adjustments as you introduce more features.

Summary

In this chapter, we embarked on creating a weather application from scratch, adhering to the TDD methodology. We employed Cypress for user acceptance tests and Jest for unit tests, progressively building out the application's features. Key practices such as modeling domain objects, stubbing network requests, and applying the single responsibility principle during refactoring were also explored.

While not exhaustive in covering all the techniques from previous chapters, this chapter underscored the importance of maintaining a disciplined pace during the development phase. It highlighted the value of being able to identify code "smells" and address them effectively, all while ensuring a solid test coverage to foster a robust and maintainable codebase. This chapter served as a practical synthesis, urging you to apply the acquired knowledge and techniques to enhance the application further.

In the upcoming and concluding chapter, we'll recap the anti-patterns we've explored, revisiting the design principles and practices we've examined, and also provide additional resources for further learning.

13
Recapping Anti-Pattern Principles

In this short final chapter, we'll briefly revisit the crucial insights from the book and furnish additional resources for you to delve deeper into the realm of React and software design.

The primary objective of this book was to unearth common anti-patterns often encountered in React code bases, especially within large-scale React applications. We delved into potential remedies and techniques to rectify these issues. The examples throughout the narrative were drawn either from my prior projects or are related to domains with which developers are likely to be familiar – such as shopping carts, user profiles, and network requests, to name a few.

I championed a step-by-step and incremental delivery approach, guiding you from an initial, less-than-ideal implementation toward a polished version, making one small improvement at a time. We embarked on organizing a typical React application, ventured into the realm of frontend testing through **test-driven development** (**TDD**), and initiated our journey with common refactoring techniques. Thereafter, we navigated the challenging waters of data/state management in React, elucidated common design principles, and explored compositional strategies. A slew of chapters were dedicated to constructing full examples from scratch, including a drop-down list, a shopping cart, and a weather application.

During this expedition, we discovered numerous handy tips, such as how to stub network requests in both Cypress and Jest, apply the strategy design pattern to a JavaScript model, and employ **Anti-Corruption Layers** (**ACLs**) in real-world code scenarios.

The techniques discussed in this book may not be groundbreaking or novel; indeed, many are well established. However, their application in the React ecosystem has remained underexplored. I earnestly hope that this book has adeptly bridged that gap, reintroducing these invaluable design principles and patterns to the React community, thereby facilitating a smoother coding experience for developers in the long term.

In this chapter, we'll revisit the following topics:

- Revisiting common anti-patterns
- Skimming through design patterns
- Revisiting foundational design principles
- Recapping techniques and practices

Revisiting common anti-patterns

We explored numerous anti-patterns in the preceding chapters. Recognizing an anti-pattern is the initial step toward rectifying it. Let's briefly recap what we've learned thus far.

Props drilling

Props drilling emerges when a prop traverses through multiple component levels, only to be employed in a deeper-level component, rendering intermediate components unnecessarily privy to this prop. This practice can lead to convoluted and hard-to-maintain code.

Solution: Employing the Context API to create a central store and functions to access this store allows the component tree to access props when needed without prop-drilling.

Long props list/big component

A component that accepts an extensive list of props or harbors a large amount of logic can become a behemoth, hard to understand, reuse, or maintain. This anti-pattern infringes upon the **Single Responsibility Principle** (**SRP**), which advocates that a component or module should only have one reason to change.

Solution: Dismantling the component into smaller, more digestible components and segregating concerns can ameliorate this issue. Each component should embody a clear, singular responsibility. Custom Hooks also serve as a potent means to simplify the code within a component and reduce its size.

Business leakage

Business leakage transpires when business logic is implanted within components that should remain purely presentational, which can complicate application management and reduce component reusability.

Solution: Extricating business logic from presentation logic using custom Hooks or relocating the business logic to a separate module or layer can address this issue. Employing an ACL can be an effective technique to rectify this issue.

Complicated logic in views

The embedding of complex logic within view components can muddle the code, making it arduous to read, comprehend, and maintain. Views should remain as uncluttered as possible, solely responsible for rendering data.

Solution: Relocating complex logic to custom Hooks, utility functions, or a separate business logic layer can help keep view components clean and manageable. Initially, breaking down components into smaller ones, and then gradually segregating the logic into appropriate places can be beneficial.

Lack of tests (at each level)

The absence of adequate unit, integration, or end-to-end tests to ascertain application functions, as anticipated, can usher in bugs, regressions, and code that's challenging to refactor or extend.

Solution: Adopting a robust testing strategy encompassing unit testing, integration testing, and end-to-end testing, coupled with practices such as TDD, can ensure code correctness and ease of maintenance.

Code duplications

Reiterating similar code across multiple components or sections of the application can complicate code-base maintenance and augment the likelihood of bugs.

Solution: Adhering to the **Don't Repeat Yourself** (**DRY**) principle and abstracting common functionality into shared utility functions, components, or Hooks can help curtail code duplication and enhance code maintainability.

Having dissected common anti-patterns, it's now imperative to delve into the design principles that act as antidotes to these prevalent issues. These principles not only provide solutions but also guide you toward writing cleaner, more efficient code.

Skimming through design patterns

There are effective patterns to counter anti-patterns in React, and interestingly, some of these patterns extend beyond the React context, being useful in broader scenarios. Let's swiftly revisit these patterns.

Higher-order components

Higher-order components (**HOCs**) are a potent pattern in React for reusing component logic. HOCs are functions that accept a component and return a new component augmented with additional properties or behaviors. By leveraging HOCs, you can extract and share common behaviors across your components, aiding in mitigating issues such as props drilling and code duplications.

Render props

The **render props** pattern encompasses a technique for sharing code between React components using a prop whose value is a function. It's a method to pass a function as a prop to a component, and that function returns a React element. This pattern can be instrumental in alleviating issues such as long props lists and big components by promoting reuse and composition.

Headless components

Headless components are those that manage behavior and logic but do not render the UI, bestowing the consumer with control over the rendering. They separate behavior logic from presentation logic, which can be a viable solution to business leakage and complicated logic in views, making components more flexible and maintainable.

Data modeling

Data modeling entails organizing and defining your data, which aids in understanding and managing the data within your application, thereby simplifying the logic within your components. This principle can be employed to tackle complicated logic in views and business logic leakage.

Layered architecture

Layered architecture involves segregating concerns and organizing code such that each layer has a specific responsibility. This separation can lead to a more organized and manageable code base, addressing issues such as business leakage and complicated logic in views.

As a reminder, *Figure 13.1* depicts this layered architecture. In a layered architecture, each layer contains numerous modules, each dedicated to specific tasks within the overall application. This includes modules for data retrieval (the **Fetcher** module, shown in *Figure 13.1*), adapters to interface with external services such as social media logins and payment gateways (the **Adaptor** gateway, shown in *Figure 13.1*), as well as components for analytics and security-related functions:

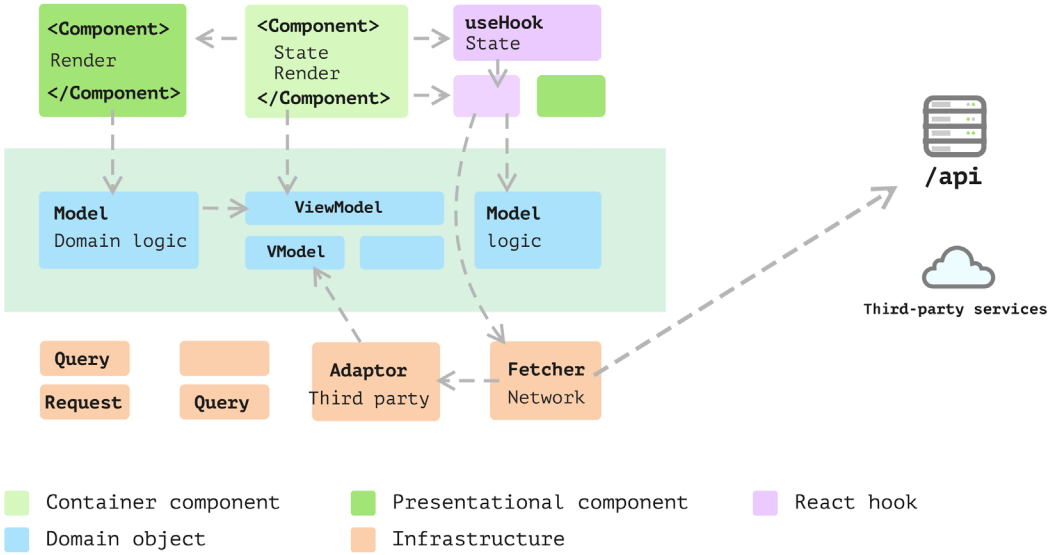


Figure 13.1: Layered architecture in a React application

We conducted a comprehensive case study on this topic in *Chapter 11*, delving deeply into the evolution of the system and pinpointing the appropriate circumstances for applying such an architectural style.

Context as an interface

Utilizing context as an interface allows components to interact with data without the need to pass props down multiple levels. This strategy can alleviate props drilling and long props lists, rendering the component tree more readable and maintainable.

With a solid grasp of foundational and design principles, it's time to explore techniques that will arm you with the practical knowledge to implement these principles in your daily coding endeavors.

Revisiting foundational design principles

Besides React-specific patterns, we've discussed several higher-level design principles throughout various chapters. These principles serve as guidelines applicable to various aspects of your work, be it React, data modeling, event testing, or scripts that facilitate integration. They are not confined to a particular context, and embracing them can significantly enhance your coding approach across different domains.

Single Responsibility Principle

The SRP advocates that a class or component should only harbor one reason to change. Adhering to the SRP can lead to more maintainable and understandable code, mitigating issues such as big components and complicated logic in views.

We've explored this principle at various levels, from carving out a smaller component from a larger one, to creating a new Hook, and up to significant refactoring such as integrating an ACL into a weather application. It's worth noting that whenever you find yourself entangled within a large component, the SRP remains your most trustworthy ally.

Dependency Inversion Principle

The **Dependency Inversion Principle** (**DIP**) emphasizes depending on abstractions, not concretions, which leads to a decoupling of high-level and low-level structures. This principle can be utilized to manage business logic leakage and promote a clean **separation of concerns** (**SoC**).

Don't Repeat Yourself

The DRY principle is about minimizing repetition within code. By adhering to the DRY principle, you can minimize code duplication, making your code base easier to maintain and extend.

Anti-Corruption Layers

An ACL serves as a barrier between different parts or layers of an application, creating a stable interface. Implementing an ACL can be a potent strategy to manage business leakage and ensure a clean SoC.

An ACL proves to be especially beneficial when your code needs to interact with other systems in any capacity, a scenario that often arises when collaborating with different teams—a common occurrence in many setups. By establishing a clear system boundary through ACL, we can mitigate the impact of changes in other systems on our own, thereby maintaining better control over our application and alleviating potential integration challenges. *Figure 13.2* illustrates how an ACL is applied in React:

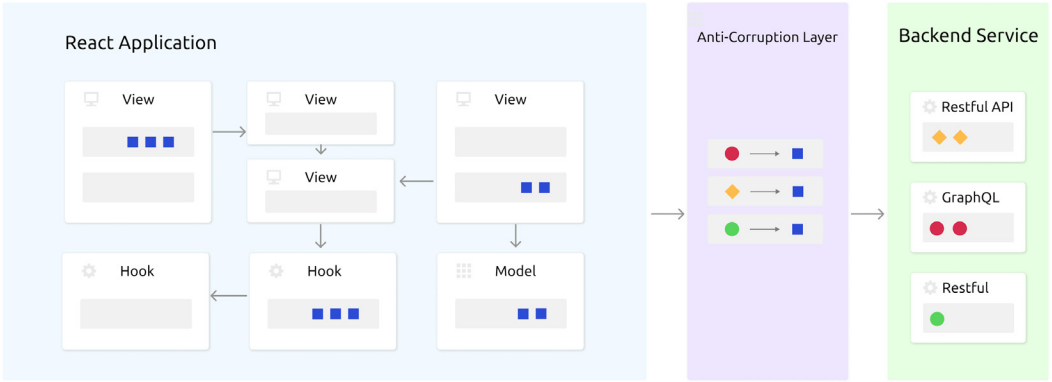


Figure 13.2: Applying an ACL in React

Using composition

Composition is a core principle in React that empowers developers to build components from other components, promoting reuse and simplicity. Employing composition can alleviate various issues, including long props lists, big components, and code duplications, leading to a more maintainable and organized code base.

Understanding these anti-patterns, design patterns, and principles is crucial for managing the complexity of a frontend code base. However, the techniques and practices are equally important as they represent the hands-on work developers engage in on a daily basis.

Recapping techniques and practices

We have placed significant emphasis on the importance of testing and making incremental improvements. This approach not only maintains high code quality but also cultivates a well-rounded developer – honing critical thinking skills and the ability to focus on solving one problem at a time.

Writing user acceptance tests

User acceptance testing (**UAT**) is a pivotal part of the development process that ensures your application aligns with its specifications and functions as desired. Implementing UAT can aid in identifying issues early in the development process, ensuring your application is on the right trajectory.

As depicted in *Figure 13.3*, we emphasize that tests should be written from the end user's perspective, focusing on delivering customer value rather than on implementation details. This is especially pertinent when you commence the implementation of a feature at a higher level:

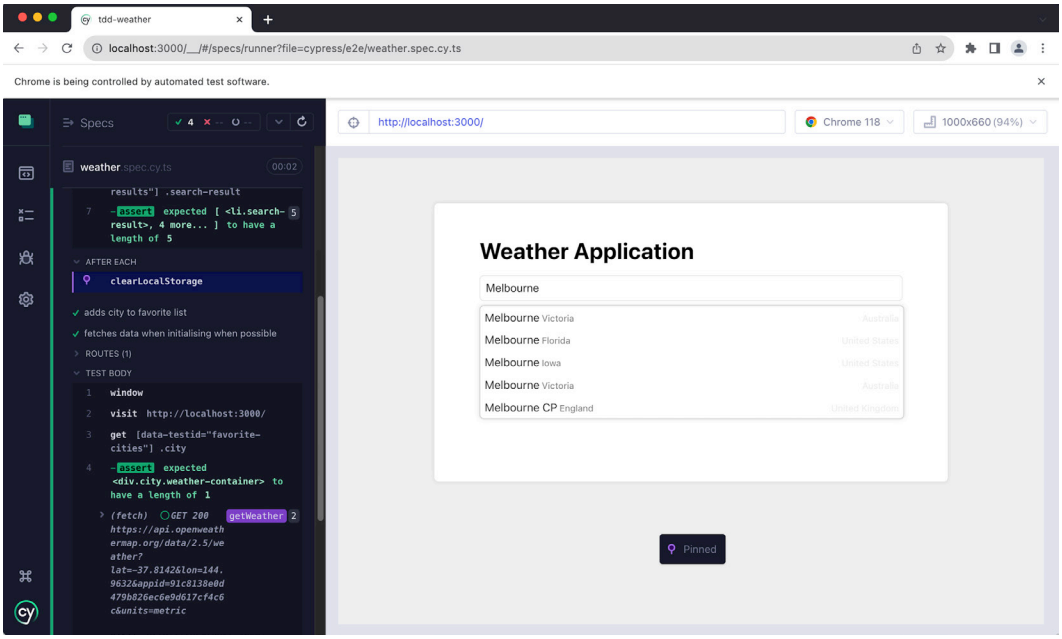


Figure 13.3: User acceptance tests

Test-Driven Development

TDD is a software engineering technique where tests are penned before code that needs to be tested. The process is primarily segmented into the following iterative development cycles: write a test, make the test pass, and then refactor. TDD can significantly help in ensuring that your code base is functional and bug-free, addressing the lack of tests at each level.

Refactoring and common code smells

Refactoring entails enhancing the design of existing code without altering its external behavior. Being cognizant of common code smells and continuously refactoring your code can lead to a healthier, more maintainable code base. This technique can be instrumental in tackling issues such as code duplication, complicated logic in views, and business leakage, among others.

Now that we've navigated through common anti-patterns, elucidated design principles, and explored techniques, it's time to look beyond this book. The following section provides a list of recommended readings that will further deepen your understanding and hone your skills in the domain of React, TypeScript, and software design principles.

Additional resources

As we draw the curtains on the contents of this book, the journey toward mastering React and avoiding common pitfalls is far from over. The landscape of web development, particularly with frameworks such as React, is ever-evolving. Continuous learning and adaptation are the keystones of staying relevant and proficient.

The following section is crafted to provide you with additional avenues for learning and exploration. These books are meticulously chosen to extend your understanding and introduce you to broader or complementary concepts in software development. Each book opens up a new dimension of knowledge, ensuring your growth trajectory remains steep and rewarding. So, as you step forward from here, let these resources be your companions in the continuous journey of learning and mastering the art of web development.

I would like to recommend a few seminal books that can further deepen your understanding and appreciation of good design, architecture, and development practices in the realm of web applications, with a particular focus on React and TypeScript:

- *Refactoring: Improving the Design of Existing Code* by Martin Fowler

  Martin Fowler's cornerstone work on refactoring is a repository of knowledge on how to enhance the structure of your code while preserving its functionality and bug-free nature. It's a must-read for anyone aspiring to hone their refactoring skills.

- *Clean Code: A Handbook of Agile Software Craftsmanship* by Robert Martin

  Robert Martin's *Clean Code* is a hallmark in the software development world. It delves into various practices and principles of penning clean, maintainable code, which is crucial for long-term success in complex projects.

- *Patterns of Enterprise Application Architecture* by Martin Fowler

  Expand your architectural vistas with this book, which dissects various patterns crucial for designing robust and scalable enterprise applications. It's a significant read to grasp the bigger picture of application architecture, extending beyond the frontend realm.

- *Test-Driven Development with React and TypeScript* by Juntao Qiu

  Immerse yourself in the world of TDD with a focus on React and TypeScript. This book navigates you through the principles of TDD and how it can markedly improve the quality, maintainability, and robustness of your code.

Summary

I want to extend my heartfelt gratitude for your dedication and passion for honing your craft and striving for technical excellence. It's individuals such as you, thirsty for knowledge and improvement, that propel our industry forward. As you turn over this last page, remember that the journey doesn't end here; in fact, it unfolds a new chapter of exploration and application in your real-world projects.

The essence of growth lies in the application and the continuous effort to apply what you've learned to challenge norms and strive for better solutions. This book aims to provide you with a solid foundation, but the real magic happens when you take these concepts, experiment with them, and integrate them into your daily work.

I sincerely appreciate your time spent traversing these pages and engaging with the material. It's my earnest wish that you carry forward this momentum, dive deeper, and continue to enrich the React community with your contributions. As you embark on the next phase of your journey, I wish you the best of luck. May your code be clean, your solutions innovative, and your journey rewarding.

Thank you, and good luck on your onward journey of continuous learning and improvement!

Index

A

abstract class 234
acceptance test-driven development (ATDD) 119
Add to Favorite feature
  current implementation, refactoring 270-276
  implementing 262-267
  multiple cities, enabling in favorite list 276-279
  weather list, refactoring 279
  weather modeling 267-270
Anti-Corruption Layer (ACL) 150, 151, 229, 285, 290
  fallback or default value, using 153-155
  implementing 259-262
  usage 151-153
anti-patterns 13, 286
  business leakage 287
  code duplications 287
  complicated logic, in views 15, 16, 287
  duplicated code 18
  in-component data transformation 14, 15
  lack of tests 16, 17, 287
  long component, with too much responsibility 18, 19
  long props list/big component 286
  props drilling 13, 14, 286
application headline
  implementing 130-132
application requirements
  breaking down 126-129
atomic design structure 50, 51
  benefits 52
  drawbacks 52
  UI components, categorizing 50

B

behavior-driven development (BDD) 87, 119
bottom-up style 126
business logic 148
business logic leak 148-150
business models
  extracting 221

C

City Search feature
implementing 251
OpenWeatherMap API 252, 253
search result list, enhancing 256-258
search results, stubbing 253-255
class-based model
benefits 230, 231
code
refactoring 140-143
Code Oven application 125, 126
enhancing 223-227
layered structure 243, 244
MenuItem, transitioning to class-based model 229, 230
MenuList, refactoring through custom hook 227, 228
code smells 292
Command Query Responsibility Segregation (CQRS) 166, 178-180, 185
commands 178
complicated logic, in views 15, 16
component 22
creating, with props 23, 24
component-based structure 49
benefits 50
drawbacks 50
component design principles
combining 72-80
Page component example 72
composition 166, 169, 291
using, to apply SRP 169-171
composition pattern 187
ExpandablePanel component, implementing 190-195
through HOCs 188
UserDashboard component example 69-72
using 69
Context API 14
using, to resolve prop drilling issue 161-164
corruption 150
Cucumber 123
Cypress 85, 95
installing 96, 97
using, in end-to-end (E2E) testing 95

D

data modeling 288
data transformation 148
Decompose Conditional
using 115, 116
Decorator design pattern 195
defensive programming 154
Dependency Inversion Principle (DIP) 166, 171, 290
applying, in analytics button 174-178
working 172, 173
describe function 89
design patterns 288
context, as interface 289
data modeling 288
headless components 288
higher-order components 288
layered architecture 288
render props 288
design principles 165
diffing algorithm 29
Document Object Model (DOM) 29
Don't Repeat Yourself (DRY) principle 18, 65-69, 151, 287, 290
Downshift 212
drop-down list component 200
developing 200-204
keyboard navigation, implementing 205-208
simplicity, maintaining 208, 209

E

end-to-end (E2E) tests 46, 83, 85
network request, intercepting 100-102
running 98-100
with Cypress 95
error boundary 11
ExpandablePanel component
implementing 190-195
Extract Constant 111
Extract Function
using 113, 114
Extract Variable
using 111, 112
Extreme Programming (XP) 120

F

feature-based structure 47, 48
benefits 48
drawbacks 49
foundational design principles 290
ACL 290
composition 291
Dependency Inversion Principle (DIP) 290
DRY principle 290
Single Responsibility Principle (SRP) 290
frontend applications
complications 45, 46

G

Gherkin 123

H

Headless Component pattern 210, 211
advantages 211
drawbacks 212
libraries 212
headless components paradigm 19, 288
Headless UI 212
higher-order components (HOCs) 18, 189, 190, 288
reviewing 188, 189
higher-order function 167
HTML (HyperText Markup Language) 4

I

in-component data transformation 14, 15
initial acceptance test
crafting 250, 251
integration tests 85, 91-95
interface-oriented programming 19
Interface Segregation Principle (ISP) 165
internal state
managing 27, 28
Introduce Parameter Object
using 114, 115
items
adding, to shopping cart 137-140

J

Jest
grouping tests 88, 89
React components, testing 89-91
simple test, writing 87
used, for writing unit tests 87
Jira 5
jsdom 95

K

kebab case 57

L

layered architecture 16, 217, 242, 243, 288
advantages 245
layered frontend application 221, 222
less-structured project
problems 44, 45
libraries, Headless Component pattern
Downshift 212
Headless UI 212
React Aria 212
React Table 212

M

maintainability 217
menu list
implementing 132-135
Move Function 132
using 116-118
multiple-component applications 219, 220
MVVM structure 52, 53
benefits 53
drawbacks 54

N

nullish coalescing 155

O

online pizza store application 125, 126
OpenWeatherMap API
key 248
optional chaining 155

P

PageProps
properties 74
Presentation Domain Data Layering
reference link 222
project structure
customized structure, exploring 58, 59
extra layer, adding to remove duplicates 55, 56
files, naming 56
files, naming with index.tsx and explicit component names 56
files, naming with kebab case 57
initial structure, implementing 54, 55
organizing 54
prop drilling issue 80
exploring 155-161
resolving, with Context API 161-164
props 22-24
components, creating with 23-24
props drilling 13, 14, 286
Puppeteer 85

Q

queries 178

R

React applications
atomic design structure 50-52
component-based structure 49, 50
evolution 218
feature-based structure 47, 48
MVVM structure 52, 53
structures 47

React Aria 212
React Context API 37-41
React Hooks 19, 27
alternative Hooks, using 197
exploring 29, 195-197
for state management 220
refactoring, for elegance and reusability 199, 200
remote data fetching 198, 199
useCallback 35-37
useEffect 31-35
useState 30, 31
ReactNode 78
React project
code base, preparing 248
React Table 212
React Testing Library 89
Red-Green-Refactor loop 120, 131
reducer function
using 182-184
refactoring 106, 107, 292
mistakes 107
tests, adding before 108-110
regression 84
remote states 6
challenges 6
Rename Variable
using 110, 111
rendering process 28, 29
DOM update 29
initial render 28
reconciliation 29
re-rendering 29
state and props changes 28
render props pattern 19, 166, 288
exploring 167-169
Replace Loop with Pipeline
using 112, 113
requirement, breaking into smaller chunks
benefits 124
restructuring 107
reusability 217

S

SearchableListContext 162
separation of concerns (SoC) 16, 217, 290
shopping cart
creating 135, 136
items, adding to 137-140
ShoppingCart component
discounts, applying to Items 233-238
implementing 231, 232
single-component applications 218, 219
Single Responsibility Principle (SRP) 19, 165, 166, 286, 290
composition, using to apply 169-171
exploring 62-64
state 27
state management 6-9
with Hooks 220
static checks 86
static component 22
Strategy pattern
exploring 238-241

T

tasking 124, 125
steps 124, 125
testability 217
test-driven development (TDD) 17, 19, 119-121, 285, 292
advantages 121
bottom-up style 126
Green stage 121
Red stage 121
Refactor stage 121
styles 122, 123
tasking 124
user value, focusing 123
testing
benefits 84
need for 84
testing types 84-86
E2E tests 85
integration tests 85
unit tests 84
test pyramid 85
top-down approach 128-130

U

UI development
difficulties 4-6
errors, thrown from other components 10, 11
unexpected user behavior 11, 12
unhappy paths 10
unit tests 84
writing, with Jest 87
useCallback hook 35-37
useEffect hook 31-35
user acceptance testing (UAT) 291
UserDashboard component
features 72
useReducer hook 180-182
user interfaces (UIs) 21
breaking down, into components 24-26
useState hook 30, 31

V

visual regression testing 86

W

weather application
previous weather data, fetching on application relaunch 280-284
requirements, reviewing 249, 250

www.packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

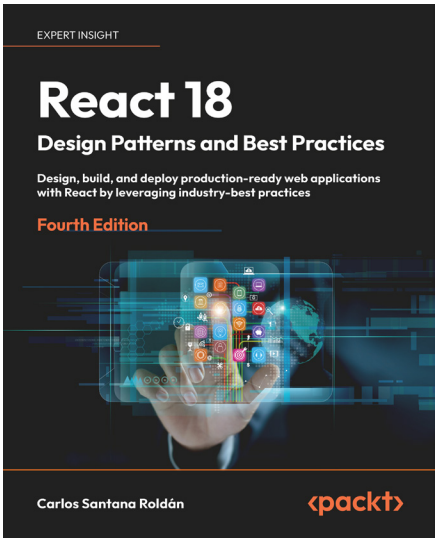
- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at packtpub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Book You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

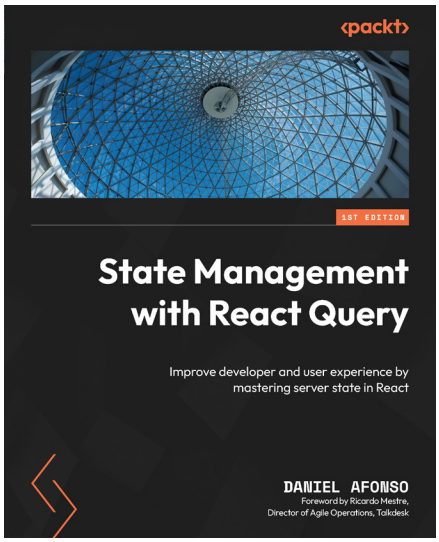


React 18 Design Patterns and Best Practices, 4e

Carlos Santana Roldán

ISBN: 978-1-80323-310-9

- Get familiar with the new React 18 and Node 19 features
- Explore TypeScript's basic and advanced capabilities
- Make components communicate with each other by applying various patterns and techniques
- Dive into MonoRepo architecture
- Use server-side rendering to make applications load faster
- Write a comprehensive set of tests to create robust and maintainable code
- Build high-performing applications by styling and optimizing React components



State Management with React Query

Daniel Afonso

ISBN: 978-1-80323-134-1

- Learn about state and how it s often managed
- Discover how state splits into server state and Client state
- Solve common challenges with server state using React Query
- Install and configure React Query and its Devtools
- Manage server state data fetching by using the useQuery hook
- Create, update and delete data by using the useMutation hook
- Discover the use of React Query with frameworks like Next.js and Remix
- Explore MSW and the Testing Library to test React Query using hooks

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit `authors.packtpub.com` and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share Your Thoughts

Now you've finished *React Anti-Patterns*, we'd love to hear your thoughts! If you purchased the book from Amazon, please click here to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below

`https://packt.link/free-ebook/9781805123972`

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly